Documento:Mongodb Con Base De Datos

Capturas De Pantalla: De Comandos Ejecutados

Nombre De Autores

Dina Arely Tino De Martínez

Marlon Gerardo Flores Portillo

Universidad Doctor Andrez Bello Reguinal Sonsonate.

Nombre: Del Módulo.

Módulo III : Bases De Datos No Relacionales

Catedrático

Ingeniero:Jose Rosalio Serrano

Sonsonate

Sábado, 16 De Mayo 2026

Capturas De Pantalla: De Comandos Ejecutados

Paso 1: Creación de la base de datos y colección

use usuario UNAB

db.clientes.insertMany([

{ nombre: "Ana", ciudad: "Lima", edad: 28, correo: "ana@gmail.com" },

{ nombre: "Carlos", ciudad: "Bogotá", edad: 34, correo: "carlos@yahoo.com" },

{ nombre: "María", ciudad: "Lima", edad: 22, correo: "maria@hotmail.com" },

{ nombre: "José", ciudad: "Quito", edad: 40, correo: "jose@gmail.com" },

{ nombre: "Lucía", ciudad: "Bogotá", edad: 19, correo: "lucia@outlook.com" },

{ nombre: "Pedro", ciudad: "Lima", edad: 35, correo: "pedro@gmail.com" }])

Captura.1

```
>_MONGOSH
> use UsuarioUNAB
< already on db UsuarioUNAB
>
>
> db["Clientes"].insertMany([
    { nombre: "Ana", ciudad: "Lima", edad: 28, correo: "ana@gmail.com" },
    { nombre: "Carlos", ciudad: "Bogotá", edad: 34, correo: "carlos@yahoo.com" },
    { nombre: "María", ciudad: "Lima", edad: 22, correo: "maria@hotmail.com" },
    { nombre: "José", ciudad: "Quito", edad: 40, correo: "jose@gmail.com" },
    { nombre: "Lucía", ciudad: "Bogotá", edad: 19, correo: "lucia@outlook.com" },
    { nombre: "Pedro", ciudad: "Lima", edad: 35, correo: "pedro@gmail.com" }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6a0650bced75f3dfa99447c6'),
      '1': ObjectId('6a0650bced75f3dfa99447c7'),
      '2': ObjectId('6a0650bced75f3dfa99447c8'),
      '3': ObjectId('6a0650bced75f3dfa99447c9'),
      '4': ObjectId('6a0650bced75f3dfa99447ca'),
      '5': ObjectId('6a0650bced75f3dfa99447cb')
    }
  }
UsuarioUNAB >
```

Paso 2: Consultas básicas y filtros

```
db.clientes.find()

db.clientes.find({ ciudad: "Lima" })

db.clientes.find({ edad: { $gt: 30 } })

db.clientes.find({ edad: { $lte: 22 } })

db.clientes.find({ ciudad: { $de: "Bogotá" } })

db.clientes.find({ ciudad: "Lima", edad: { $gt: 25 } })
```

Captura 2.

```
}
db["Clientes"].find()

db["Clientes"].find({ ciudad: "Lima" })

db["Clientes"].find({ edad: { $gt: 30 } })

db["Clientes"].find({ edad: { $lte: 22 } })

db["Clientes"].find({ ciudad: { $ne: "Bogotá" } })

db["Clientes"].find({ ciudad: "Lima", edad: { $gt: 25 } })
```

Captura 2.

```
< {
    _id: ObjectId('6a0650bced75f3dfa99447c6'),
    nombre: 'Ana',
    ciudad: 'Lima',
    edad: 28,
    correo: 'ana@gmail.com'
  }
  {
    _id: ObjectId('6a0650bced75f3dfa99447cb'),
    nombre: 'Pedro',
    ciudad: 'Lima',
    edad: 35,
    correo: 'pedro@gmail.com'
  }
```

Parte 3: Operadores lógicos

db.clientes.find({ ciudad: { $in: ["Lima", "Bogotá"] } })

db.clientes.find({ ciudad: { $nin: ["Lima", "Quito"] } })

db.clientes.find({ $or: [{ edad: { $lt: 20 } }, { ciudad: "Quito" }] })

captura 3.

```
> db["clientes"].find({ ciudad: { $in: ["Lima", "Bogotá"] } })
  db["clientes"].find({ ciudad: { $nin: ["Lima", "Quito"] } })
  db["clientes"].find({ $or: [ { edad: { $lt: 20 } }, { ciudad: "Quito" } ] })
< {
    _id: ObjectId('6a062f59647b5de08bbf5620'),
    nombre: 'José',
    ciudad: 'Quito',
    edad: 40,
    correo: 'jose@gmail.com'
  }
  {
    _id: ObjectId('6a062f59647b5de08bbf5621'),
    nombre: 'Lucía',
    ciudad: 'Bogotá',
    edad: 19,
    correo: 'lucia@outlook.com'
  }
UsuarioUNAB>
```

Parte 4: Proyección de campos

```
db.clientes.find({}, { nombre: 1, ciudad: 1, _id: 0 })
```

db.clientes.find({}, { correo: 0 })

```
> db["clientes"].find({}, { nombre: 1, ciudad: 1, _id: 0 })
  db["clientes"].find({}, { correo: 0 })
< {
    _id: ObjectId('6a062f59647b5de08bbf561d'),
    nombre: 'Ana',
    ciudad: 'Lima',
    edad: 28
  }
  {
    _id: ObjectId('6a062f59647b5de08bbf561e'),
    nombre: 'Carlos',
    ciudad: 'Bogotá',
    edad: 34
  }
```

```
}
{
  _id: ObjectId('6a062f59647b5de08bbf561f'),
  nombre: 'María',
  ciudad: 'Lima',
  edad: 22
}
{
  _id: ObjectId('6a062f59647b5de08bbf5620'),
  nombre: 'José',
  ciudad: 'Quito',
  edad: 40
}
{
  _id: ObjectId('6a062f59647b5de08bbf5621'),
  nombre: 'Lucía',
  ciudad: 'Bogotá',
  edad: 19
```

```
}
{
  _id: ObjectId('6a062f59647b5de08bbf5622'),
  nombre: 'Pedro',
  ciudad: 'Lima',
  edad: 35
}
```

Parte 5: Ordenamiento y límites

db.clientes.find().sort({ edad: 1 })

db.clientes.find().sort({ edad: -1 })

db.clientes.find().limit(3)

db.clientes.find().skip(2)

Captura.5

```
{
  _id: ObjectId('6a062f59647b5de08bbf5621'),
  nombre: 'Lucía',
  ciudad: 'Bogotá',
  edad: 19,
  correo: 'lucia@outlook.com'
}
{
  _id: ObjectId('6a062f59647b5de08bbf5622'),
  nombre: 'Pedro',
  ciudad: 'Lima',
  edad: 35,
  correo: 'pedro@gmail.com'
}
suarioUNAB>
```

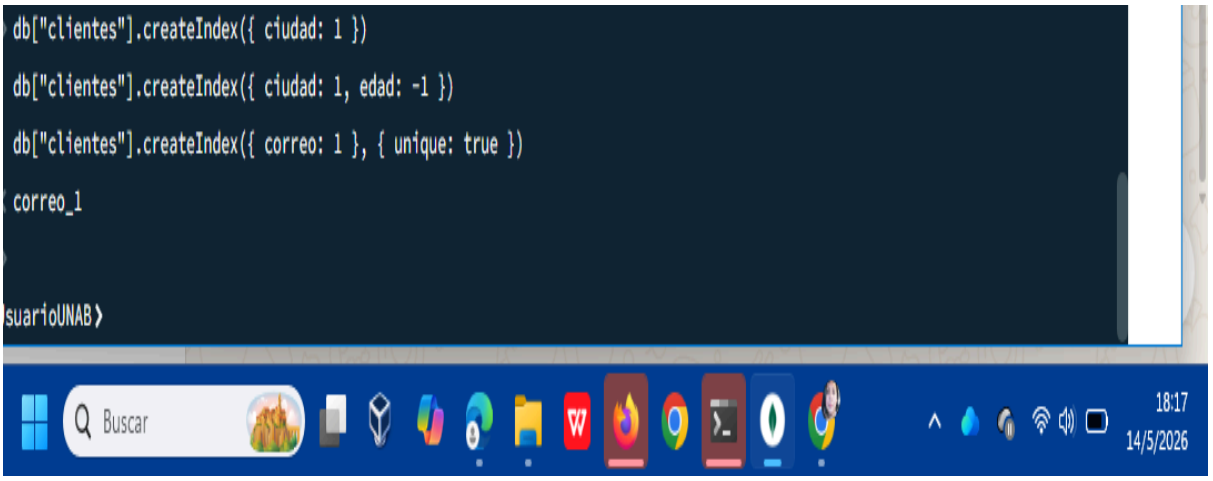
Parte 6. Índices

db.clientes.create Index({ ciudad: 1 })

db.clientes.create Index({ ciudad: 1, edad: -1 })

db.clientes.create Index({ correo: 1 }, { unique: true })

Captura 6.



Que hace cada comando

1. createIndex({ ciudad: 1 }) - Crea un índice ascendente por ciudad para que las búsquedas por ciudad sean más rápidas

2. createIndex({ ciudad: 1, edad: -1 }) - Índice compuesto: primero por ciudad ascendente, luego por edad descendente

3. createIndex({ correo: 1 }, { unique: true }) - Hace que el campo correo no pueda tener valores repetidos

Parte 7: Análisis de rendimiento con .explain()

db.clientes.find({ nombre: "Pedro" }).explain("executionStats")

db.clientes.create Index({ nombre: 1 })

db.clientes.find({ nombre: "Pedro" }).explain("executionStats")

captura 7.Nota: Lo que muestra captura de pantalla

1. Comando ejecutado:Qué db["clientes"].find({ nombre: "Pedro" }).explain("execution Stats")

2. Resultado clave:

stage: 'FETCH'

input Stage: {

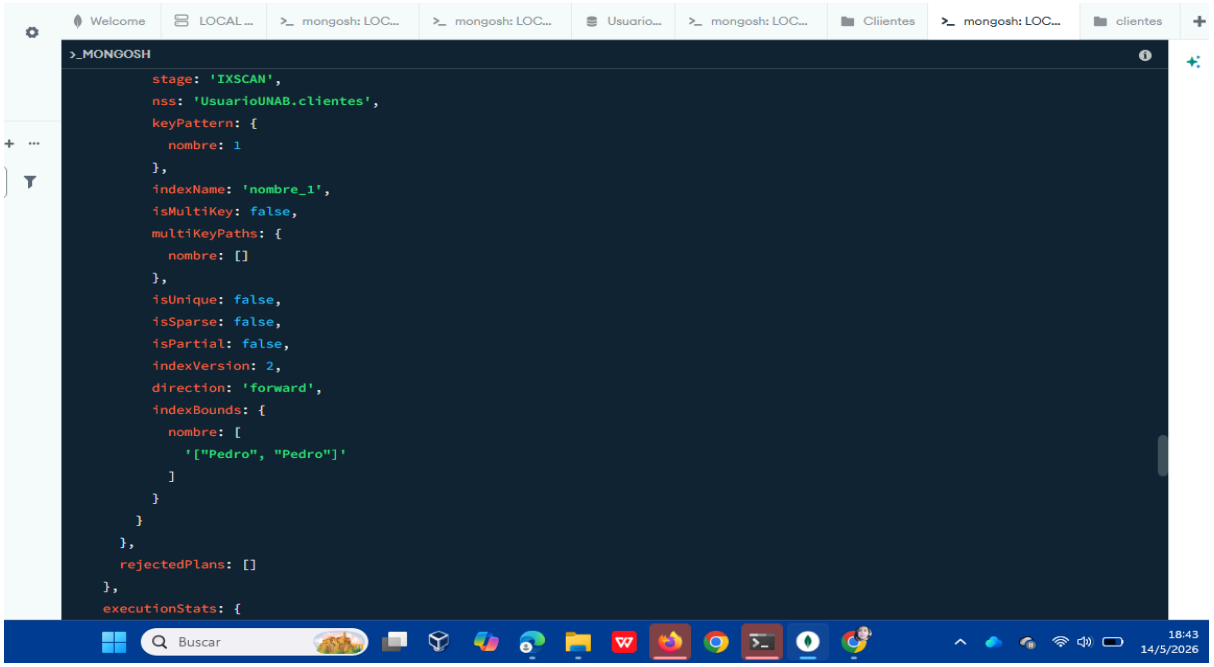
  stage: 'IXCAN'

}

Qué significa eso:

- IX SCAN = Index Scan. MongoDB usó el índice que creaste en nombre para buscar a Pedro. No tuvo que revisar toda la colección.

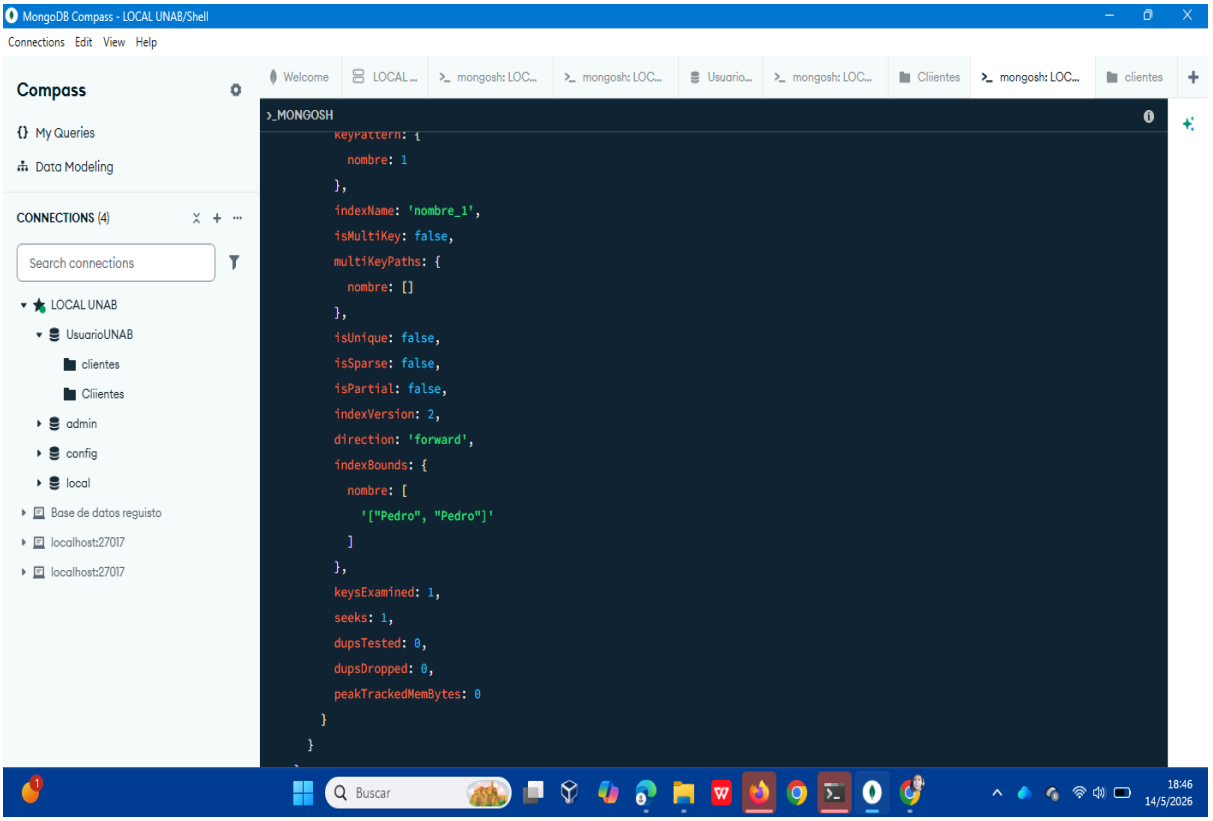
- FETCH = Después de encontrar la posición en el índice, va a traer el documento completo.

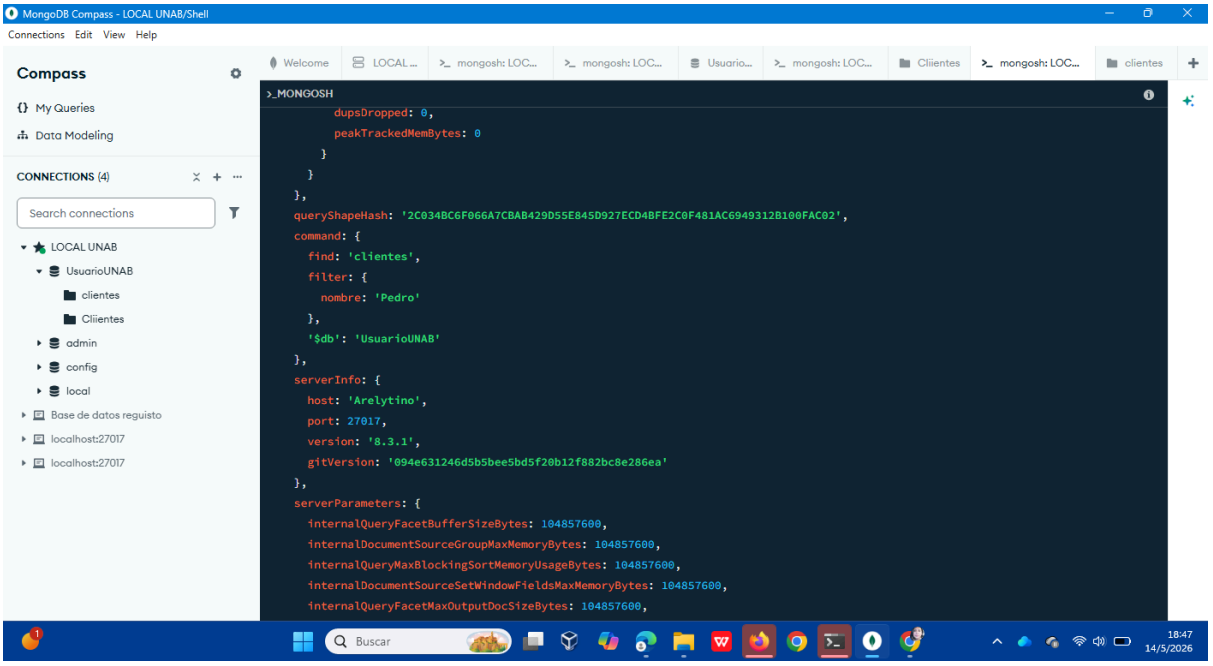


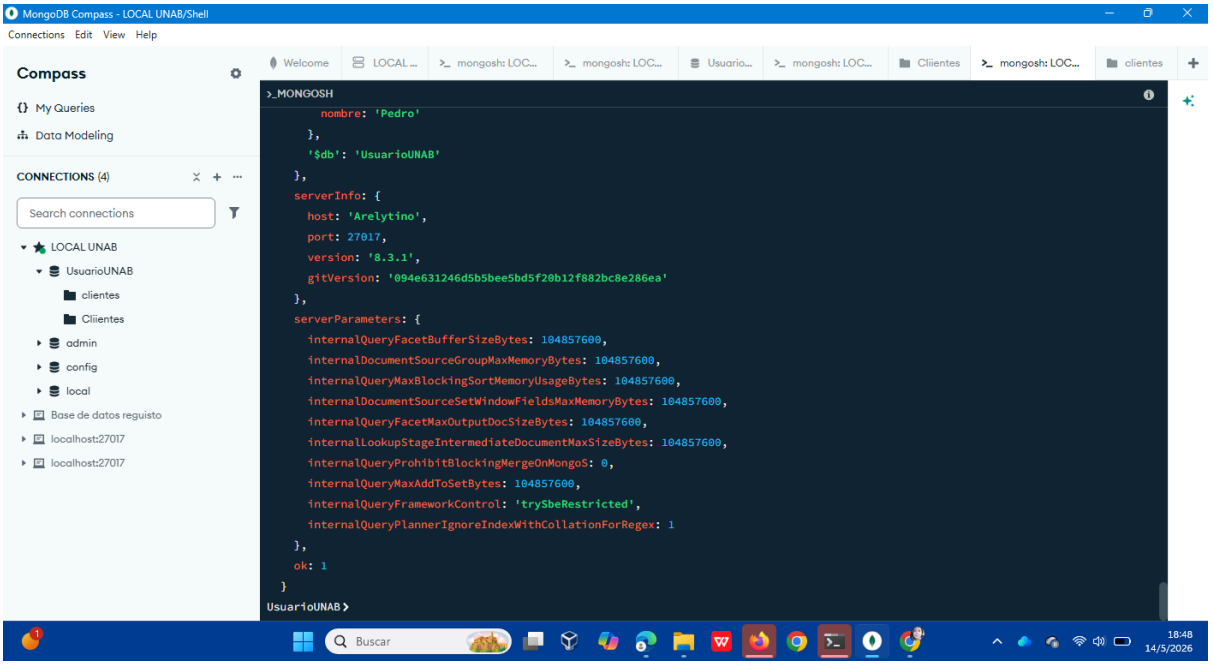
```
>_MONGOSH
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      nss: 'UsuarioUNAB.clientes',
      inputStage: {
        stage: 'IXSCAN',
        nss: 'UsuarioUNAB.clientes',
        keyPattern: {
          nombre: 1
        },
        indexName: 'nombre_1',
        isMultiKey: false,
        multiKeyPaths: {
          nombre: []
        },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          nombre: [
            '["Pedro", "Pedro"]'
          ]
        }
```

```
>_MONGOSH
          nombre: [
            '["Pedro", "Pedro"]'
          ]
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 12,
    totalKeysExamined: 1,
    totalDocsExamined: 1,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 1,
      executionTimeMillisEstimate: 5,
      works: 2,
      advanced: 1,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
```

```
    restoreState: 0,
    isEOF: 1,
    nss: 'UsuarioUNAB.clientes',
    docsExamined: 1,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 1,
      executionTimeMillisEstimate: 5,
      works: 2,
      advanced: 1,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      nss: 'UsuarioUNAB.clientes',
      keyPattern: {
        nombre: 1
      },
      indexName: 'nombre_1',
      isMultiKey: false,
```







Parte 8: Eliminación de índices

db.clientes.getIndexes()

db.clientes.drop Index("nombre 1")

captura 8.En la captura se ve:

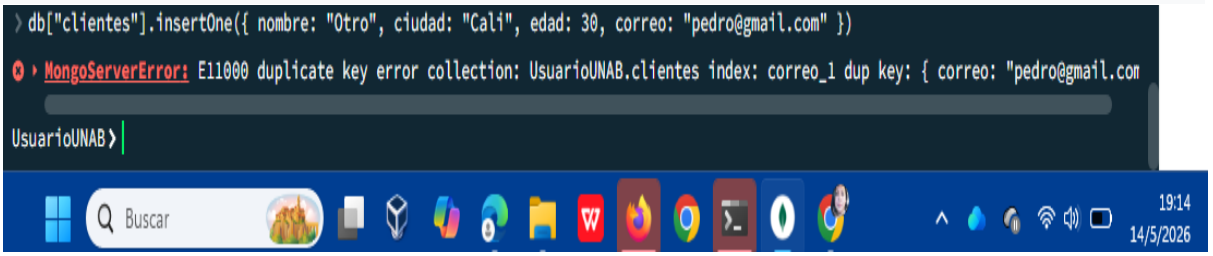
db["clientes"].drop Index("nombre 1")

{ nIndexesWas: 5, ok: 1 }

Qué significa:

- nIndexesWas: 5 = Antes de borrar había 5 índices en la colección clientes

- ok: 1 = Se ejecutó correctamente, el índice nombre_1 ya no existe

Parte 9: Validación

db.clientes.insertOne({ nombre: "Otro", ciudad: "Cali", edad: 30, correo: "pedro@gmail.com" })